Side effect (computer science)

In computer science, an operation or expression is said to have a **side effect** if it has any observable effect other than its primary effect of reading the value of its arguments and returning a value to the invoker of the operation. Example side effects include modifying a non-local variable, a static local variable or a mutable argument passed by reference; performing I/O; or calling other functions with side-effects.[1] In the presence of side effects, a program's behaviour may depend on history; that is, the order of evaluation matters. Understanding and debugging a function with side effects requires knowledge about the context and its possible histories.[2][3] Side effects play an important role in the design and analysis of programming languages. The degree to which side effects are used depends on the programming paradigm. For example, imperative programming is commonly used to produce side effects, to update a system's state. By contrast, declarative programming is commonly used to report on the state of system, without side effects.

Functional programming aims to minimize or eliminate side effects. The lack of side effects makes it easier to do formal verification of a program. The functional language Haskell eliminates side effects such as I/O and other stateful computations by replacing them with monadic actions.[4][5] Functional languages such as Standard ML, Scheme and Scala do not restrict side effects, but it is customary for programmers to avoid them.[6]

Effect systems extend types to keep track of effects, permitting concise notation for functions with effects, while maintaining information about the extent and nature of side effects. In particular, functions without effects correspond to pure functions.

Assembly language programmers must be aware of *hidden* side effects—instructions that modify parts of the processor state which are not mentioned in the instruction's mnemonic. A classic example of a hidden side effect is an arithmetic instruction that implicitly modifies condition codes (a hidden side effect) while it explicitly modifies a register (the intended effect). One potential drawback of an instruction set with hidden side effects is that, if many instructions have side effects on a single piece of state, like condition codes, then the logic required to update that state sequentially may become a performance bottleneck. The problem is particularly acute on some processors designed with pipelining (since 1990) or with out-of-order execution. Such a processor may require additional control circuitry to detect hidden side effects and stall the pipeline if the next instruction depends on the results of those effects.

Referential transparency

Absence of side effects is a necessary, but not sufficient, condition for referential transparency. Referential transparency means that an expression (such as a function call) can be replaced with its value. This requires that the expression is pure, that is to say the expression must be deterministic (always give the same value for the same input) and side-effect free.

Temporal side effects

Side effects caused by the time taken for an operation to execute are usually ignored when discussing side effects and referential transparency. There are some cases, such as with hardware timing or testing, where operations are inserted specifically for their temporal side effects e.g. `sleep(5000)` or `for (int i = 0; i < 10000; ++i) {}`. These instructions do not change state other than taking an amount of time to complete.

Idempotence

A subroutine with side effects is idempotent if multiple applications of the subroutine have the same effect on the system state as a single application, in other words if the function from the system state space to itself associated with the subroutine is idempotent in the mathematical sense. For instance, consider the following Python program:

```
x = 0

def setx(n):
    global x
    x = n

setx(3)
assert x == 3
setx(3)
assert x == 3
```

`setx` is idempotent because the second application of `setx` to 3 has the same effect on the system state as the first application: `x` was already set to 3 after the first application, and it is still set to 3 after the second application.

A pure function is idempotent if it is idempotent in the mathematical sense. For instance, consider the following Python program:

```
def abs(n):
    return -n if n < 0 else n

assert abs(abs(-3)) == abs(-3)
```

`abs` is idempotent because the second application of `abs` to the return value of the first application to -3 returns the same value as the first application to -3.

Example

One common demonstration of side effect behavior is that of the assignment operator in C. The assignment `a = b` is an expression that evaluates to the same value as the expression `b`, with the side effect of storing the R-value of `b` into the L-value of `a`. This allows multiple assignment:

```
a = (b = 3);  // b = 3 evaluates to 3, which then gets assigned to a
```

Because the operator right associates, this is equivalent to

```
a = b = 3;
```

This presents a potential hangup for novice programmers who may confuse

```
while (b == 3) {}  // tests if b evaluates to 3
```

with

```
while (b = 3) {}  // b = 3 evaluates to 3, which then casts to true so the loop is infinite
```

See also

- Action at a distance (computer programming)
- Don't-care term
- Effect system
- Monad (functional programming)
- Sequence point
- Side-channel attack
- Undefined behaviour
- Unspecified behaviour
- Frame problem

References

1. Spuler, David A.; Sajeev, A. Sayed Muhammed (January 1994). *Compiler Detection of Function Call Side Effects*. James Cook University. CiteSeerX 10.1.1.70.2096 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.70.2096). "The term Side effect refers to the modification of the nonlocal environment. Generally this happens when a function (or a procedure) modifies a global variable or arguments passed by reference parameters. But here are other ways in which the nonlocal environment can be modified. We consider the following causes of side effects through a function call: 1. Performing I/O. 2. Modifying global variables. 3. Modifying local permanent variables (like static variables in C). 4. Modifying an argument passed by reference. 5. Modifying a local variable, either automatic or static, of a function higher up in the function call sequence (usually via a pointer)."
2. Turner, David A., ed. (1990). *Research Topics in Functional Programming*. Addison-Wesley. pp. 17–42. Via Hughes, John. "Why Functional Programming Matters" (http://www.cs.utexas.edu/~shmat/courses/cs345/whyfp.pdf) (PDF). Archived (https://web.archive.org/web/20220614042648/https://www.cs.utexas.edu/~shmat/courses/cs345/whyfp.pdf) (PDF) from the original on 2022-06-14. Retrieved 2022-08-06.
3. Collberg, Christian S. (2005-04-22). "CSc 520 Principles of Programming Languages" (https://web.archive.org/web/20220806155136/https://www2.cs.arizona.edu/~collberg/Teaching/520/2005/Html/Html-24/index.html). Department of Computer Science, University of Arizona.

Archived from the original (http://www.cs.arizona.edu/~collberg/Teaching/520/2005/Html/Html-24/index.html) on 2022-08-06. Retrieved 2022-08-06.

4. "Haskell 98 report" (http://www.haskell.org). 1998.
5. Jones, Simon Peyton; Wadler, Phil (1993). *Imperative Functional Programming*. Conference Record of the 20th Annual ACM Symposium on Principles of Programming Languages. pp. 71–84.
6. Felleisen, Matthias; Findler, Robert Bruce; Flatt, Matthew; Krishnamurthi, Shriram (2014-08-01). "How To Design Programs" (http://www.htdp.org/) (2 ed.). MIT Press.
